

Study Report: QA & Test Engineer - Career Path & Core Skills

Written by Gagan Pasupuleti

Family:	QA & Test Engineer
Level:	Beginner to Advanced
Tracks:	Beginner, Intermediate, Advanced
Chapters:	18
Source:	Official docs, free libraries, and industry-standard references (see sources and books)

Official sources and free libraries

- <https://docs.pytest.org/>
- <https://playwright.dev/python/docs/intro>
- <https://www.selenium.dev/documentation/>
- <https://learning.postman.com/docs/>
- <https://www.istqb.org/>
- <https://docs.github.com/en/actions>

Summary

Quality assurance engineering covering manual test design, pytest automation, Selenium and Playwright browser testing, Postman API validation, CI/CD gates, and ISTQB-aligned test management practices for web and API products shipping in 2024-2026.

Skill stack by level

Beginner

- Software testing fundamentals and SDLC
- Test case design (equivalence partitioning, boundary values)
- Bug reporting with reproduction steps
- Manual exploratory testing
- Browser DevTools for network and console inspection
- Basic SQL for test data validation
- Git for test artifact versioning

Intermediate

- pytest fixtures and parametrization
- Page Object Model with Selenium WebDriver
- Playwright auto-waiting and trace viewer

- Postman collections and Newman CLI
- API contract testing and JSON Schema validation
- CI integration (GitHub Actions, Jenkins)
- Test data factories and environment management
- Accessibility smoke checks with axe

Advanced

- Performance testing basics (k6, Locust)
- Visual regression testing
- Test strategy and risk-based prioritization
- ISTQB test planning and metrics (defect density, escape rate)
- Parallel test execution and flaky test triage
- Security testing fundamentals (OWASP Top 10 awareness)
- Shift-left QA in agile squads

Recommended books

1. Explore It! - Elisabeth Hendrickson

Level: Beginner | Session-based exploratory testing and charter design.

2. The Art of Software Testing - Glenford Myers et al.

Level: Intermediate | Test design, equivalence partitioning, and defect prevention.

3. Agile Testing - Lisa Crispin & Janet Gregory

Level: Intermediate | Testing in Scrum: whole-team quality and automation strategy.

4. Continuous Delivery - Jez Humble & David Farley

Level: Advanced | CI/CD pipelines, test automation gates, and deployment safety.

5. ISTQB Foundation Level Syllabus - ISTQB

Level: Beginner | Industry-standard testing terminology and fundamentals (free PDF).

End-to-end projects

Project 1: Manual Test Suite for Web App

Beginner | 1-2 weeks - Write 30 test cases, execute exploratory sessions, log bugs in a tracking template.

- Plan: Test plan PDF
- Cases: Test case spreadsheet
- Execute: Execution log
- Report: Test summary report

Project 2: Automated API + UI Test Suite

Intermediate | 3-4 weeks - pytest API tests + Playwright E2E for login flow; run in GitHub Actions on every PR.

- API: API test suite
- E2E: E2E video recordings
- CI: CI badge green
- Report: Test dashboard HTML

Project 3: Performance + Security Test Program

Advanced | 4-5 weeks - k6 load test plan, OWASP ZAP scan, and combined quality gate report for a web application.

- Load: k6 HTML report

- Security: ZAP report
- Gate: Gate checklist
- Present: Sign-off presentation

Chapters

Track: Beginner

Chapter 1: QA Role in Modern Software Delivery [Beginner]

Chapter focus: QA Role in Modern Software Delivery *(Level: Beginner)**

QA engineers protect users by finding defects before production and verifying fixes after. Quality is whole-team responsibility; QA leads test strategy and automation advocacy. Shift-left means involving QA during requirements to catch ambiguities early. Your goal is risk reduction informed by business impact, not bug counting alone.

Code Reference:

Bug Report Template:

- Title: Checkout fails when coupon expired
- Steps: 1) Add item 2) Apply EXPIRED10 3) Pay
- Expected: Error message
- Actual: 500 error
- Severity: High | Priority: P1

What it shows: Structured bug report gives developers reproduction path without back-and-forth.

Topic guide

What it actually is

QA validates software meets requirements and is fit for release.

When to use it

Engage from story refinement through release verification.

Where to use it

Agile squads, release trains, and regulated industries with audit trails.

Real use example

QA blocks release when payment regression found two hours before go-live.

Chapter 2: Test Case Design Techniques [Beginner]

Chapter focus: Test Case Design Techniques *(Level: Beginner)**

Equivalence partitioning groups inputs expected to behave same-test one per group. Boundary value analysis targets edges: min, max, min-1, max+1. Decision tables map combinations of

conditions to expected outcomes systematically. Each test case needs preconditions, steps, expected result, and traceability to requirement ID.

Code Reference:

```
# BVA example: age field accepts 18-65
# Test: 17 (invalid), 18 (valid), 65 (valid), 66 (invalid)
```

What it shows: Boundary tests catch off-by-one defects common in validation logic.

Topic guide

What it actually is

Systematic test design maximizes defect detection per execution hour.

When to use it

Apply when specifying tests for forms, pricing rules, and date ranges.

Where to use it

Banking limits, e-commerce promotions, and insurance quote calculators.

Real use example

Coupon field rejects 17-character code because max length 16 caught at boundary test.

Chapter 3: Manual Exploratory Testing [Beginner]

Chapter focus: Manual Exploratory Testing** *(Level: Beginner)

Exploratory testing combines learning, test design, and execution simultaneously. Charter defines mission: Explore checkout with invalid payment methods for 90 minutes. Note anomalies, questions, and risks-not just pass/fail binary results. Session-based test management tracks time boxes and findings for reporting.

Code Reference:

```
Charter: Explore user profile avatar upload with oversized and wrong-type files.
Findings: .exe renamed to .png accepted; no size limit; EXIF not stripped.
```

What it shows: Charter focuses session; findings list concrete issues discovered.

Topic guide

What it actually is

Exploratory testing uncovers unexpected defects scripted cases miss.

When to use it

Run on new features, after major refactors, and before release candidates.

Where to use it

Every sprint demo, beta program, and post-automation gap analysis.

Real use example

Explorer finds race condition when double-clicking Pay only during slow 3G throttle.

Chapter 4: Browser DevTools for Testers [Beginner]

Chapter focus: Browser DevTools for Testers** *(Level: Beginner)

Network tab reveals failed API calls, status codes, and payload shapes. Console shows JavaScript errors invisible to users but breaking functionality. Application tab inspects cookies, localStorage, and service workers. Throttle network to simulate mobile conditions during manual passes.

Code Reference:

```
// Console quick check after checkout
performance.getEntriesByType('navigation')[0].duration
// Network: filter Fetch/XHR, inspect POST /api/orders response
```

What it shows: Navigation timing measures page load; XHR filter isolates API failures.

Topic guide

What it actually is

DevTools let testers diagnose frontend issues without developer intervention.

When to use it

Use on every bug investigation and API-integrated UI test session.

Where to use it

Web apps, SPAs, and hybrid mobile wrappers.

Real use example

Tester proves 401 caused by missing Authorization header using Network tab HAR export.

Chapter 5: SQL Basics for Test Data Validation [Beginner]

Chapter focus: SQL Basics for Test Data Validation** *(Level: Beginner)

Testers query databases to verify backend state matches UI claims. SELECT with WHERE confirms row created; COUNT validates batch imports. Never run UPDATE/DELETE on production-use read-only accounts in prod investigations. Sanitized staging copies mirror schema for safe destructive validation.

Code Reference:

```
SELECT id, status, total FROM orders WHERE user_id = 42 ORDER BY created_at DESC LIMIT 5;
```

What it shows: Query confirms latest orders for test user match dashboard display.

Topic guide

What it actually is

SQL verification cross-checks UI against source of truth in database.

When to use it

Use after API or UI actions affecting persistent data.

Where to use it

Order status, subscription billing, and inventory reconciliation tests.

Real use example

UI shows 'Shipped' but SQL still 'PAID'-bug filed with query evidence attached.

Chapter 6: Bug Lifecycle and Severity vs Priority [Beginner]

Chapter focus: Bug Lifecycle and Severity vs Priority *(Level: Beginner)**

Severity measures user/business impact; priority measures fix order for team capacity. Critical severity crash on login may be priority P1; cosmetic typo low severity P3. Bug lifecycle: New ? Assigned ? Fixed ? Retest ? Closed or Reopened. Include environment, build version, and attachments in every report.

Code Reference:

```
Severity: Critical - payment double-charged  
Priority: P1 - fix before any other sprint work  
Status: Reopened after failed retest on build 1.2.4
```

What it shows: Reopened status tracks regression when fix did not hold.

Topic guide

What it actually is

Bug taxonomy communicates impact and urgency to engineering and product.

When to use it

Apply consistent definitions across team documented in QA wiki.

Where to use it

Jira, Linear, GitHub Issues, and release go/no-go meetings.

Real use example

P1 payment bug stops release; P3 tooltip fix deferred to next sprint.

Track: Intermediate

Chapter 7: pytest Automation Framework Structure [Intermediate]

Chapter focus: pytest Automation Framework Structure *(Level: Intermediate)**

conftest.py shares fixtures across test packages without import boilerplate. Arrange-Act-Assert

keeps tests readable; one logical assertion focus per test when possible. Markers (@pytest.mark.smoke) slice suites for fast PR gates vs full nightly runs. pytest.param combines inputs with test ids for clear failure output.

Code Reference:

```
import pytest

@pytest.mark.parametrize('email,expected', [
    ('user@example.com', 201),
    ('not-an-email', 422),
    ('', 422),
], ids=['valid', 'invalid-format', 'empty'])
def test_register_status(api_client, email, expected):
    assert api_client.post('/register', json={'email': email}).status_code == expected
```

What it shows: parametrize runs three cases; ids label each in failure reports.

Topic guide

What it actually is

pytest is the standard Python test runner for API and unit automation.

When to use it

Adopt for all Python-based test automation in backend-heavy teams.

Where to use it

CI smoke suites, microservice contract tests, and data pipeline validation.

Real use example

Smoke marker runs in 3 minutes on PR; full regression runs nightly in 45 minutes.

Chapter 8: Selenium WebDriver and Page Object Model [Intermediate]

Chapter focus: Selenium WebDriver and Page Object Model** *(Level: Intermediate)

Selenium drives browsers programmatically; WebDriver W3C standardizes API across languages. Page Object Model encapsulates locators and actions in page classes reducing duplication. Explicit waits (WebDriverWait) handle async UI better than sleep(5) flakes. Run headless in CI; headed locally for debugging selector issues.

Code Reference:

```
class LoginPage:
    def __init__(self, driver):
        self.driver = driver
    def login(self, email, password):
        self.driver.find_element(By.ID, 'email').send_keys(email)
        self.driver.find_element(By.ID, 'password').send_keys(password)
        self.driver.find_element(By.CSS_SELECTOR, 'button[type=submit]').click()
```

What it shows: LoginPage hides locator details; tests call login() with credentials only.

Topic guide

What it actually is

Selenium automates browser UI regression at scale with mature ecosystem.

When to use it

Use when Playwright not available or legacy Java/C# grid infrastructure exists.

Where to use it

Enterprise regression grids, cross-browser corporate apps, and SAP web UIs.

Real use example

Locator change updates LoginPage once; fifty tests keep passing unchanged.

Chapter 9: Playwright Modern E2E Testing [Intermediate]

Chapter focus: Playwright Modern E2E Testing** *(Level: Intermediate)

Playwright auto-waits for element actionability reducing flaky explicit wait code. Trace viewer records DOM, network, and screenshots step-by-step on failure. Built-in test generator records actions producing starter locators-refine to getByRole. Python sync API suits pytest teams; async available for advanced flows.

Code Reference:

```
def test_search_product(page):
    page.goto('https://demo.shop')
    page.get_by_label('Search').fill('keyboard')
    page.get_by_role('button', name='Search').click()
    expect(page.get_by_role('heading', name='Results')).to_be_visible()
```

What it shows: get_by_label and get_by_role prefer accessible selectors over brittle CSS.

Topic guide

What it actually is

Playwright is the preferred 2024-2026 browser automation for new E2E suites.

When to use it

Start new E2E projects with Playwright unless constrained by existing Selenium grid.

Where to use it

Cross-browser CI, visual debug, and mobile viewport emulation.

Real use example

Failed CI uploads trace.zip; QA replays exact click sequence seeing network 500.

Chapter 10: Postman and Newman API Testing [Intermediate]

Chapter focus: Postman and Newman API Testing** *(Level: Intermediate)

Postman collections group requests with tests scripts asserting status and JSON body. Environment variables switch base URL and tokens between dev, staging, prod read-only. Newman runs collections headless in CI with reporters (JUnit, HTML). Contract tests fail when API response schema drifts from documented shape.

Code Reference:

```
pm.test('Status is 200', () => pm.response.to.have.status(200));
pm.test('Has items array', () => pm.expect(pm.response.json().items).to.be.an('array'));
```

What it shows: pm.test assertions run after each request in collection runner or Newman.

Topic guide

What it actually is

Postman/Newman validates APIs independently of UI for faster feedback.

When to use it

Run API regression on every backend deploy before UI E2E.

Where to use it

Microservices, mobile backends, and third-party integration verification.

Real use example

Newman fails CI when pagination field renamed breaking mobile app contract.

Chapter 11: CI/CD Test Gates in GitHub Actions [Intermediate]

Chapter focus: CI/CD Test Gates in GitHub Actions** *(Level: Intermediate)

Quality gates block merge when lint, unit, API, or E2E jobs fail. Parallel jobs split API and UI suites for faster feedback. Artifacts store reports, traces, and screenshots for failed run debugging. Required checks enforce policy; optional jobs warn without blocking early adoption.

Code Reference:

```
jobs:
  api-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt
      - run: pytest tests/api --junitxml=report.xml
      - uses: actions/upload-artifact@v4
        with: { name: junit, path: report.xml }
```

What it shows: JUnit XML uploads integrate test results into GitHub Checks UI.

Topic guide

What it actually is

CI pipelines automate test execution on every code change.

When to use it

Integrate smoke tests on PR; full regression on main nightly.

Where to use it

GitHub Actions, GitLab CI, Azure DevOps, and Jenkins pipelines.

Real use example

Merge blocked until API smoke passes; developer fixes regression same day.

Chapter 12: Test Data Management and Factories [Intermediate]**Chapter focus: Test Data Management and Factories** *(Level: Intermediate)**

Factories generate unique emails and SKUs preventing collision in parallel runs. Seed scripts reset staging to known baseline before nightly regression. Never use production PII in tests-synthetic data only with GDPR compliance. Teardown deletes created records via API keeping database lean.

Code Reference:

```
def user_factory():
    uid = uuid.uuid4().hex[:8]
    return {'email': f'test_{uid}@example.com', 'password': 'Str0ng!pass'}
```

What it shows: UUID suffix guarantees unique email across parallel pytest workers.

Topic guide**What it actually is**

Test data strategy prevents flaky collisions and compliance violations.

When to use it

Define before running parallel CI or shared staging environments.

Where to use it

Automated suites, load tests, and demo environment refresh scripts.

Real use example

Parallel shard 3 no longer fails from duplicate email unique constraint violation.

Chapter 13: Accessibility Smoke Testing [Intermediate]**Chapter focus: Accessibility Smoke Testing** *(Level: Intermediate)**

WCAG 2.2 guidelines define perceivable, operable, understandable, robust UI. axe-core integrates with Playwright flagging missing labels and contrast failures. Manual keyboard-only pass catches focus trap bugs automation misses. Accessibility defects are release blockers for public sector and many enterprise contracts.

Code Reference:

```
accessibility_scan = await page.accessibility.snapshot()
```

```
# Playwright + axe: expect(await axe.run(page)).to_have_no_violations()
```

What it shows: Automated axe scan lists violations with selector and WCAG rule id.

Topic guide

What it actually is

Accessibility testing ensures products usable by people with disabilities.

When to use it

Run automated axe on critical pages each sprint; full audit quarterly.

Where to use it

Government portals, education platforms, and EU Accessibility Act scope apps.

Real use example

Checkout page fails axe on low contrast button; fixed before legal review.

Track: Advanced

Chapter 14: Performance Testing with k6 [Advanced]

Chapter focus: Performance Testing with k6** *(Level: Advanced)

Performance testing validates latency and throughput under expected and peak load. k6 scripts define VUs (virtual users) ramping scenarios in JavaScript. Thresholds fail CI when P95 exceeds SLA (e.g., `http_req_duration p(95)<500ms`). Test staging only with approval-never DDoS production inadvertently.

Code Reference:

```
import http from 'k6/http';
import { check } from 'k6';
export default function () {
  const res = http.get('https://staging.example/api/products');
  check(res, { 'status 200': (r) => r.status === 200 });
}
```

What it shows: Virtual user iteration hits products endpoint; check validates status.

Topic guide

What it actually is

Load testing reveals scalability bottlenecks before users do during peaks.

When to use it

Run before major sales events and after infrastructure changes.

Where to use it

E-commerce flash sales, election result pages, and API gateway sizing.

Real use example

k6 finds DB connection pool exhausted at 200 VUs; pool size increased pre-launch.

Chapter 15: ISTQB Test Planning and Metrics [Advanced]**Chapter focus: ISTQB Test Planning and Metrics** *(Level: Advanced)**

Test plan documents scope, approach, resources, schedule, and exit criteria. Entry criteria define when testing starts; exit criteria define release readiness. Defect density (bugs/KLOC) and escape rate (prod bugs/release) track quality trends. Risk-based testing prioritizes effort on high-impact, high-likelihood areas.

Code Reference:**Exit Criteria Example:**

- 100% critical/high test cases executed
- Zero open P1 defects
- <=2 open P2 with business sign-off
- Automation smoke pass on release candidate build

What it shows: Exit criteria make go/no-go objective rather than subjective gut feel.

Topic guide**What it actually is**

Formal test planning aligns QA activity with business risk tolerance.

When to use it

Required in regulated domains; valuable everywhere releases are frequent.

Where to use it

Medical devices, fintech, and enterprise RFP-driven projects.

Real use example

Release board approves ship when exit criteria met; one waived P2 documented.

Chapter 16: Flaky Test Triage and Quarantine [Advanced]**Chapter focus: Flaky Test Triage and Quarantine** *(Level: Advanced)**

Flaky tests pass and fail without code changes-eroding trust in CI. Quarantine flaky tests (@pytest.mark.quarantine) off critical path while fixing root cause. Common causes: timing, shared state, external dependency instability, clock sensitivity. Track flake rate metric; zero tolerance on main branch smoke suite.

Code Reference:

```
# Retry only in CI with pytest-rerunfailures
# @pytest.mark.flaky(reruns=2) - temporary, must ticket root cause
```

What it shows: Reruns mitigate temporarily but root cause fix required for permanent trust.

Topic guide

What it actually is

Flake management keeps CI signal trustworthy for release decisions.

When to use it

Implement quarantine policy when E2E suite exceeds 50 tests.

Where to use it

Large Playwright/Selenium suites in high-velocity teams.

Real use example

Race on toast notification fixed with `expect().to_be_visible()` replacing `sleep`.

Chapter 17: Security Testing Awareness (OWASP) [Advanced]

Chapter focus: Security Testing Awareness (OWASP)** *(Level: Advanced)

OWASP Top 10 lists injection, broken auth, XSS, SSRF as common web risks. QA performs lightweight security tests: SQL injection strings in forms, XSS in rich text. Report security findings privately; do not exploit beyond proof-of-concept. Coordinate with security team for DAST/SAST tool interpretation.

Code Reference:

```
# Negative test payload examples (use only in authorized staging)
payloads = ['" OR '1'='1", '<script>alert(1)</script>', '../../etc/passwd']
```

What it shows: Payload list used systematically in authorized staging negative tests.

Topic guide

What it actually is

Security-aware QA catches obvious vulnerabilities before penetration test.

When to use it

Include security negative cases in regression for auth and input-heavy features.

Where to use it

Login, search, file upload, and admin panels.

Real use example

Reflected XSS in search query blocked after QA files P1 before external pen test.

Chapter 18: Shift-Left QA in Agile Squads [Advanced]

Chapter focus: Shift-Left QA in Agile Squads** *(Level: Advanced)

Shift-left embeds QA in refinement, Three Amigos, and definition of done. Testability requirements (data-testid, feature flags) negotiated before development starts. Automation candidates identified during story splitting-not after feature ships. QA coaches developers on unit

test coverage for complex business rules.

Code Reference:

Definition of Done:

- AC verified by QA
- Automation added or ticket filed
- No open P1/P2
- Release notes updated

What it shows: DoD checklist prevents 'done' stories missing test and documentation work.

Topic guide

What it actually is

Shift-left reduces cost of defect fix by catching issues before code merge.

When to use it

Adopt when sprint velocity suffers from end-sprint test bottlenecks.

Where to use it

Scrum teams, kanban flow, and continuous delivery organizations.

Real use example

Bug found in refinement saves eight hours vs same bug found in production.